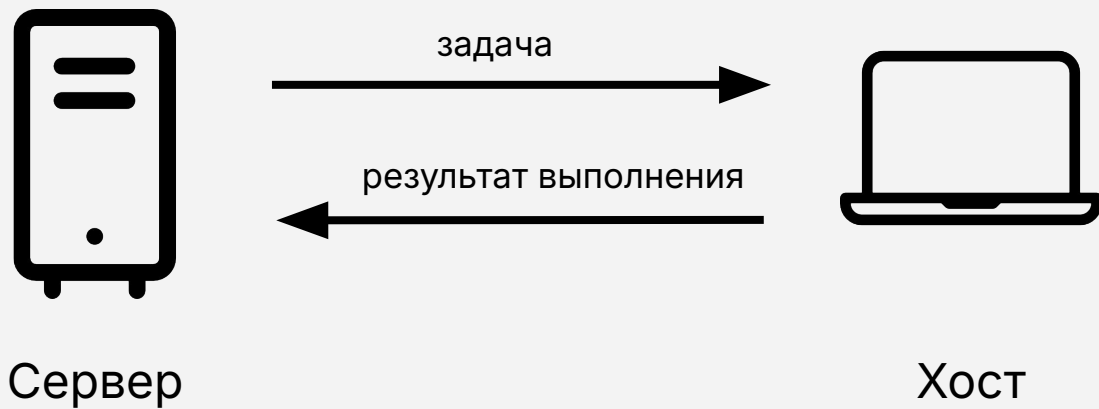
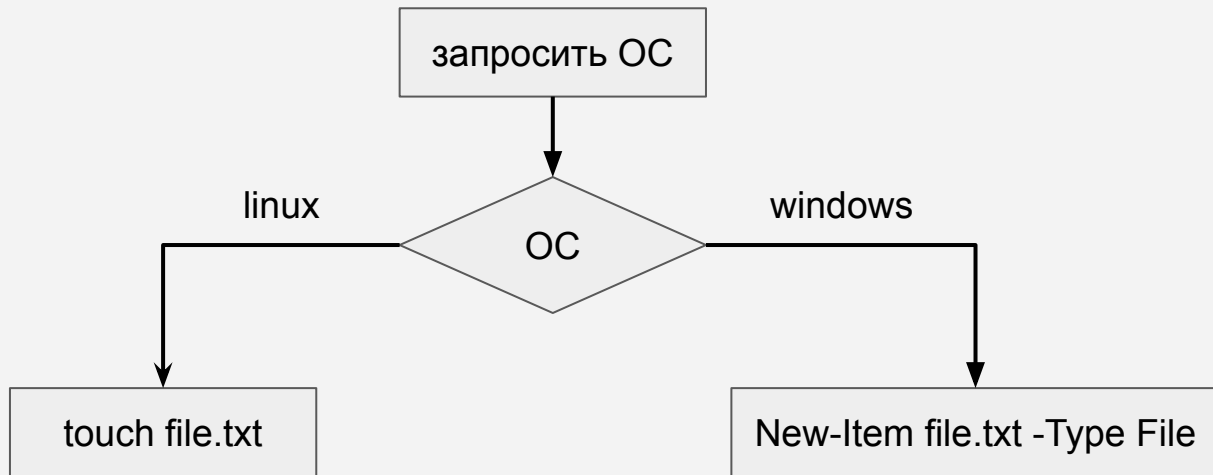


Генераторы в Python

Предыстория



Создание файла



Предыстория

```
def create_file():  
    runner_id = gen_random_uuid()  
    request_os(runner_id)  
    os = yield runner_id  
    if os == "linux":  
        request_new_file_linux()  
    else:  
        request_new_file_windows()
```

```
import dill  
  
def start_algo():  
    gen = create_file()  
    runner_id = next(gen)  
    gen_bytes = dill.dumps(gen)  
    db.insert(runner_id, gen_bytes)  
  
def continue_algo(runner_id, result):  
    gen_bytes = db.select(runner_id)  
    gen = dill.loads(gen_bytes)  
    gen.send(result)
```

Предыстория

```
~~~~~  
File "/Users/gleb-pp/Downloads/venv/lib/python3.14/site-packages/dill/_dill.py", line 423, in save  
    StockPickler.save(self, obj, save_persistent_id)
```

```
~~~~~  
File "/Library/Frameworks/Python.framework/Versions/3.14/lib/python3.14/pickle.py", line 590, in save  
    rv = reduce(self.proto)
```

TypeError: cannot pickle 'generator' object

when serializing dict item 'gen'
when serializing Runner state
when serializing Runner object

Предыстория

```
File "/Users/gleb-pp/Downloads/venv/lib/python3.14/site-packages/dill/_dill.py", line 423, in save
    StockPickler.save(self, obj, save_persistent_id)
File "/Library/Frameworks/Python.framework/Versions/3.14/lib/python3.14/pickle.py", line 590, in save
    rv = reduce(self.proto)
TypeError: cannot pickle 'generator' object
when serializing dict item 'gen'
when serializing Runner state
when serializing Runner object
```

Генераторы в Python не сериализуемы!

Но почему? Это ведь объект в памяти!

План

- **Введение:** Что такое генератор? Как Python определяет генератор?
- **Объект генератора в CPython:** Структуры PyGenObject и PyInterpreterFrame
- **Процесс работы генератора:** Что происходит во время yield, next(), send()?
- **Эволюция:** Что было до генераторов? Вложенные генераторы. Корутины
- **Бенчмарки и границы применимости**
 - генератор vs список
 - генератор vs функция
 - генератор vs итератор-класс
 - накладные расходы на вложенность

Введение

Что такое генератор?

Генератор — особый случай итератора, который автоматически сохраняет состояние выполнения (фрейм) между вызовами `next()` и `send()`.

```
def gen():
    print("Hello, World!")
    a = yield 0
    print(f"Received: {a}")
    b = yield 1
    print(f"Received: {b}")

g = gen()
print(next(g))      # print "Hello, World!" and yield 0
print(g.send(10))   # print "Received: 10" and yield 1
print(g.send(20))   # print "Received: 20" and raise StopIteration
```

Генератор vs. Функция

При вызове функции f():

- создаётся frame
- код исполняется
- возвращается значение
- frame уничтожается

```
def f():  
    return 10  
  
print(f())  # 10
```

При вызове генератора g():

- создается объект-генератор
- на frame выделяется память
- frame пока не инициализируется

```
def g():  
    yield 10  
  
print(g())  # <generator object g at 0x104a4e800>
```

Как Python определяет генератор?

Когда компилятор встречает `yield`, функция получает флаг `CO_GENERATOR`

```
def f():  
    return 10  
  
def gen():  
    yield 10  
  
CO_GENERATOR = 0x0020 # 32  
print(f"f() это генератор: bool(f.__code__.co_flags & CO_GENERATOR)}") # False  
print(f"gen() это генератор: bool(gen.__code__.co_flags & CO_GENERATOR)}") # True
```

`CO_GENERATOR` — сигнал о том, что при вызове необходимо создать не обычный `frame-return`, а `generator object`.

Генератор в CPython

Структура PyGenObject

```
struct PyGenObject {
    PyObject *gi_weakreflist;           // список слабых ссылок на генератор (weakref)
    PyObject *gi_name;                  // имя генератора
    PyObject *gi_qualname;              // полное имя (с учетом вложенности)
    _PyErr_StackItem gi_exc_state;      // состояние исключений между yield-ами
    PyObject gi_origin_or_finalizer;    // origin / finalizer

    // флаги
    char gi_hooks_initd;                // инициализированы ли хуки (firstiter / finalizer)
    char gi_closed;                     // закрыт ли генератор?
    char gi_running_async;               // выполняется ли сейчас async-генератор?

    int8_t gi_frame_state;              // состояние фрейма
    _PyInterpreterFrame gi_iframe;      // встроенный фрейм (сердце генератора!)
};
```

Структура PyGenObject

```
struct PyGenObject {
    PyObject *gi_weakreflist;           // список слабых ссылок на генератор (weakref)
    PyObject *gi_name;                  // имя генератора
    PyObject *gi_qualname;              // полное имя (с учетом вложенности)
    _PyErr_StackItem gi_exc_state;      // состояние исключений между yield-ами
    PyObject gi_origin_or_finalizer;   // origin / finalizer

    // флаги
    char gi_hooks_initd;                // инициализированы ли хуки (firstiter / finalizer)
    char gi_closed;                     // закрыт ли генератор?
    char gi_running_async;              // выполняется ли сейчас async-генератор?

    int8_t gi_frame_state;              // состояние фрейма
    _PyInterpreterFrame gi_iframe;      // встроенный фрейм (сердце генератора!)
};
```

gi_origin_or_finalizer

Поле `gi_origin_or_finalizer` может использоваться в трех режимах:

- в обычных генераторах — `Null`

gi_origin_or_finalizer

Поле `gi_origin_or_finalizer` может использоваться в трех режимах:

- в обычных генераторах — `Null`
- корутины (`async def` без `yield`) — `origin` (откуда создан генератор)

```
import sys
import asyncio
sys.set_coroutine_origin_tracking_depth(2)

async def my_coro():
    await asyncio.sleep(0)

coro = my_coro()
print(coro.cr_origin)
# (('Users/gleb-pp/Documents/Projects/cpython/check.py', 8, '<module>'),)
```


gi_origin_or_finalizer

Поле `gi_origin_or_finalizer` может использоваться в трех режимах:

- в обычных генераторах — `Null`
- корутины (`async def` без `yield`) — `origin` (откуда создан генератор)
- асинхронные генераторы (`async def` + `yield`) — `finalizer` (вызывается при закрытии генератора)

```
import sys

def my_finalizer(agen):
    print(f"Финализация {agen}")

sys.set_asyncgen_hooks(finalizer=my_finalizer)

async def my_agen():
    yield 1
```

Структура PyGenObject

```
struct PyGenObject {
    PyObject *gi_weakreflist;           // список слабых ссылок на генератор (weakref)
    PyObject *gi_name;                  // имя генератора
    PyObject *gi_qualname;              // полное имя (с учетом вложенности)
    _PyErr_StackItem gi_exc_state;      // состояние исключений между yield-ами
    PyObject gi_origin_or_finalizer;    // origin / finalizer

    // флаги
    char gi_hooks_initd;                // инициализированы ли хуки (firstiter / finalizer)
    char gi_closed;                     // закрыт ли генератор?
    char gi_running_async;               // выполняется ли сейчас async-генератор?

    int8_t gi_frame_state;             // состояние фрейма
    _PyInterpreterFrame gi_iframe;      // встроенный фрейм (сердце генератора!)
};
```

gi_frame_state

Состояние генератора:

- FRAME_CREATED: Фрейм выделен, но не инициализирован
- FRAME_SUSPENDED: Фрейм приостановлен на yield
- FRAME_SUSPENDED_YIELD_FROM: Фрейм приостановлен на yield from
- FRAME_EXECUTING: Фрейм выполняется прямо сейчас
- FRAME_CLEARED: Фрейм завершен, очищен, памяти нет

Структура PyGenObject

```
struct PyGenObject {
    PyObject *gi_weakreflist;           // список слабых ссылок на генератор (weakref)
    PyObject *gi_name;                  // имя генератора
    PyObject *gi_qualname;              // полное имя (с учетом вложенности)
    _PyErr_StackItem gi_exc_state;      // состояние исключений между yield-ами
    PyObject gi_origin_or_finalizer;    // origin / finalizer

    // флаги
    char gi_hooks_initd;                // инициализированы ли хуки (firstiter / finalizer)
    char gi_closed;                     // закрыт ли генератор?
    char gi_running_async;               // выполняется ли сейчас async-генератор?

    int8_t gi_frame_state;              // состояние фрейма
    _PyInterpreterFrame gi_iframe;     // встроенный фрейм (сердце генератора!)
};
```

Структура PyInterpreterFrame

```
struct _PyInterpreterFrame {
    _PyStackRef f_executable;           // ссылка на исполняемый код (PyCodeObject)
    struct _PyInterpreterFrame *previous; // связный список фреймов (стек вызовов)
    _PyStackRef f_funcobj;              // функция/объект, который выполняется
    PyObject *f_globals;                // глобальные переменные
    PyObject *f_builtins;               // встроенные функции и типы (print, len, open)
    PyFrameObject *frame_obj;           // Python-объект фрейма (inspect.currentframe())
    PyObject *f_locals;                 // Python-объект локальных переменных
    _Py_CODEUNIT *instr_ptr;            // указатель на текущую инструкцию в байткоде
    _PyStackRef *stackpointer;          // указатель на вершину стека
    uint16_t return_offset;             // куда вернуться после return
    char owner;                         // кто владеет фреймом (генератор / поток / объект фрейма)
    uint8_t visited;                   // флаг для отладки и GC (для обхода графа объектов)
    _PyStackRef localsplus[1];          // массив локальных переменных и стека
};
```

Процесс работы генератора

Инициализация генератора

- `gen` уже имеет флаг `CO_GENERATOR` (от компилятора)
- при вызове `get()` создается объект `PyGenObject`
- `gi_frame_state = FRAME_CREATED`
- выделена память на фрейм, но он пока не инициализирован

```
PyObject *PyGen_New(PyFrameObject *f) {  
    PyGenObject *gen = PyObject_GC_NewVar(...);  
    gen->gi_frame_state = FRAME_CREATED;  
    return (PyObject *)gen;  
}
```

```
def gen():  
    x = 10  
    y = yield x  
    yield x + y
```

```
g = gen()  
a = next(g)  
b = g.send(5)  
c = next(g)
```

Первый вызов `next(g)`

```
def gen():  
    x = 10  
    y = yield x  
    yield x + y
```

```
g = gen()  
a = next(g)  
b = g.send(5)  
c = next(g)
```


Первый вызов `next(g)`

- проверка, не завершен ли генератор

```
if (FRAME_STATE_FINISHED(frame_state)) {  
    // ...  
}
```

```
def gen():  
    x = 10  
    y = yield x  
    yield x + y
```

```
g = gen()  
a = next(g)  
b = g.send(5)  
c = next(g)
```

Первый вызов `next(g)`

- проверка, не завершен ли генератор
- проверка, не выполняется ли генератор сейчас

```
if (frame_state == FRAME_EXECUTING) {  
    gen_raise_already_executing_error(gen);  
}
```

```
def gen():  
    x = 10  
    y = yield x  
    yield x + y
```

```
g = gen()  
a = next(g)  
b = g.send(5)  
c = next(g)
```

Первый вызов `next(g)`

- проверка, не завершен ли генератор
- проверка, не выполняется ли генератор сейчас
- перевод состояния генератора в `FRAME_EXECUTING`

```
gen->gi_frame_state = FRAME_EXECUTING;
```

```
def gen():  
    x = 10  
    y = yield x  
    yield x + y
```

```
g = gen()  
a = next(g)  
b = g.send(5)  
c = next(g)
```

Первый вызов `next(g)`

- проверка, не завершен ли генератор
- проверка, не выполняется ли генератор сейчас
- перевод состояния генератора в `FRAME_EXECUTING`
- установка исполняемого кода, `instr_ptr` (на `x = 10`), `stackpointer`, ссылки на предыдущий фрейм, владельца

```
_PyInterpreterFrame *frame = &gen->gi_iframe;  
PyCodeObject *code = _PyGen_GetCode(gen);  
frame->f_executable = PyStackRef_FromPyObjectNew((PyObject *)code);  
frame->instr_ptr = _PyCode_GetFirstInstr(code);  
frame->stackpointer = frame->localsplus;  
frame->previous = _PyThreadState_GET()->current_frame;  
frame->owner = FRAME_OWNED_BY_GENERATOR;
```

```
def gen():  
    x = 10  
    y = yield x  
    yield x + y
```

```
g = gen()  
a = next(g)  
b = g.send(5)  
c = next(g)
```

Первый вызов `next(g)`

- проверка, не завершен ли генератор
- проверка, не выполняется ли генератор сейчас
- перевод состояния генератора в `FRAME_EXECUTING`
- установка исполняемого кода, `instr_ptr` (на `x = 10`), `stackpointer`, ссылки на предыдущий фрейм, владельца
- сохранение переменной в `localsplus[0]`

```
def gen():  
    x = 10  
    y = yield x  
    yield x + y
```

```
g = gen()  
a = next(g)  
b = g.send(5)  
c = next(g)
```

Первый вызов next (g)

```
// загрузка константы 10
case TARGET(Load_Const): {
    PyObject *value = GETCONST(oparg);
    *stackpointer++ = PyStackRef_FromPyObjectNew(value);
    instr_ptr++;
    break;
}
// сохранение в localsplus[0]
case TARGET(Store_Fast): {
    PyObject *value = PyStackRef_AsPyObjectBorrow(PEEK(-1));
    frame->localsplus[oparg] = PyStackRef_FromPyObjectNew(value);
    stackpointer--;
    instr_ptr++;
    break;
}
// загрузка x (10) на стек
case TARGET(Load_Fast): {
    int index = oparg;
    *stackpointer++ = frame->localsplus[index];
    instr_ptr++;
    break;
}
```

```
def gen():
    x = 10
    y = yield x
    yield x + y
```

```
g = gen()
a = next(g)
b = g.send(5)
c = next(g)
```

Первый вызов `next(g)`

- проверка, не завершен ли генератор
- проверка, не выполняется ли генератор сейчас
- перевод состояния генератора в `FRAME_EXECUTING`
- установка исполняемого кода, `instr_ptr` (на `x = 10`), `stackpointer`, ссылки на предыдущий фрейм, владельца
- сохранение переменной в `localsplus[0]`
- изменение `instr_ptr` на инструкцию после `YIELD_VALUE`

```
frame->instr_ptr = instr_ptr + 1;
```

```
def gen():  
    x = 10  
    y = yield x  
    yield x + y
```

```
g = gen()  
a = next(g)  
b = g.send(5)  
c = next(g)
```

Первый вызов `next(g)`

- проверка, не завершен ли генератор
- проверка, не выполняется ли генератор сейчас
- перевод состояния генератора в `FRAME_EXECUTING`
- установка исполняемого кода, `instr_ptr` (на `x = 10`), `stackpointer`, ссылки на предыдущий фрейм, владельца
- сохранение переменной в `localsplus[0]`
- изменение `instr_ptr` на инструкцию после `YIELD_VALUE`
- перевод состояния генератора в `FRAME_SUSPENDED`

```
def gen():  
    x = 10  
    y = yield x  
    yield x + y
```

```
g = gen()  
a = next(g)  
b = g.send(5)  
c = next(g)
```

```
gen->gi_frame_state = FRAME_SUSPENDED;
```


Первый вызов `next(g)`

- проверка, не завершен ли генератор
- проверка, не выполняется ли генератор сейчас
- перевод состояния генератора в `FRAME_EXECUTING`
- установка исполняемого кода, `instr_ptr` (на `x = 10`), `stackpointer`, ссылки на предыдущий фрейм, владельца
- сохранение переменной в `localsplus[0]`
- изменение `instr_ptr` на инструкцию после `YIELD_VALUE`
- перевод состояния генератора в `FRAME_SUSPENDED`
- возврат результата

```
def gen():  
    x = 10  
    y = yield x  
    yield x + y
```

```
g = gen()  
a = next(g)  
b = g.send(5)  
c = next(g)
```

Вызов `send()`

- проверка, не завершен ли генератор
- проверка, не выполняется ли генератор сейчас

```
def gen():  
    x = 10  
    y = yield x  
    yield x + y
```

```
g = gen()  
a = next(g)  
b = g.send(5)  
c = next(g)
```

Вызов `send()`

- проверка, не завершен ли генератор
- проверка, не выполняется ли генератор сейчас
- перевод состояния генератора в `FRAME_EXECUTING`

```
gen->gi_frame_state = FRAME_EXECUTING;
```

```
def gen():  
    x = 10  
    y = yield x  
    yield x + y
```

```
g = gen()  
a = next(g)  
b = g.send(5)  
c = next(g)
```

Вызов `send()`

- проверка, не завершен ли генератор
- проверка, не выполняется ли генератор сейчас
- перевод состояния генератора в `FRAME_EXECUTING`
- помещение аргумента `send` на стек фрейма

```
_PyFrame_StackPush(frame, PyStackRef_FromPyObjectNew(arg));
```

```
def gen():  
    x = 10  
    y = yield x  
    yield x + y
```

```
g = gen()  
a = next(g)  
b = g.send(5)  
c = next(g)
```

Вызов `send()`

- проверка, не завершен ли генератор
- проверка, не выполняется ли генератор сейчас
- перевод состояния генератора в `FRAME_EXECUTING`
- помещение аргумента `send` на стек фрейма
- продолжение функции с сохраненного `instr_ptr`
- сложение, в результате которого 15 кладется на стек
- изменение `instr_ptr` на инструкцию после `YIELD_VALUE`
- перевод состояния генератора в `FRAME_SUSPENDED`

```
def gen():  
    x = 10  
    y = yield x  
    yield x + y
```

```
g = gen()  
a = next(g)  
b = g.send(5)  
c = next(g)
```

Завершение

- проверка, не завершен ли генератор
- проверка, не выполняется ли генератор сейчас
- перевод состояния генератора в `FRAME_EXECUTING`

```
def gen():  
    x = 10  
    y = yield x  
    yield x + y
```

```
g = gen()  
a = next(g)  
b = g.send(5)  
c = next(g)
```

Завершение

- проверка, не завершен ли генератор
- проверка, не выполняется ли генератор сейчас
- перевод состояния генератора в `FRAME_EXECUTING`
- помещение `None` на стек фрейма

```
PyObject *arg_obj = arg ? arg : Py_None;  
_PyFrame_StackPush(frame, PyStackRef_FromPyObjectNew(arg_obj));
```

```
def gen():  
    x = 10  
    y = yield x  
    yield x + y
```

```
g = gen()  
a = next(g)  
b = g.send(5)  
c = next(g)
```

Завершение

- проверка, не завершен ли генератор
- проверка, не выполняется ли генератор сейчас
- перевод состояния генератора в `FRAME_EXECUTING`
- помещение `None` на стек фрейма
- перевод состояния генератора в `FRAME_CLEARED`

```
gen->gi_frame_state = FRAME_CLEARED;
```

```
def gen():  
    x = 10  
    y = yield x  
    yield x + y
```

```
g = gen()  
a = next(g)  
b = g.send(5)  
c = next(g)
```


Завершение

- проверка, не завершен ли генератор
- проверка, не выполняется ли генератор сейчас
- перевод состояния генератора в `FRAME_EXECUTING`
- помещение `None` на стек фрейма
- перевод состояния генератора в `FRAME_CLEARED`
- очистка фрейма (удаление локальных переменных и стека)

```
def gen():  
    x = 10  
    y = yield x  
    yield x + y
```

```
g = gen()  
a = next(g)  
b = g.send(5)  
c = next(g)
```

Завершение

```
_PyInterpreterFrame *frame = &gen->gi_iframe;

// очищаем все локальные переменные на стеке
_PyStackRef *sp = frame->stackpointer;
while (sp > frame->localsplus) {
    // уменьшаем счётчики ссылок
    PyStackRef_CLEAR(--sp);
}

// обнуляем указатели
frame->stackpointer = NULL;
frame->instr_ptr = NULL;
frame->previous = NULL;

// очищаем f_locals (Python-словарь)
Py_CLEAR(frame->f_locals);

// очищаем frame_obj (Python-объект фрейма)
if (frame->frame_obj) {
    Py_CLEAR(frame->frame_obj);
}

// очищаем f_executable (код)
PyStackRef_CLEAR(frame->f_executable);
```

```
def gen():
    x = 10
    y = yield x
    yield x + y
```

```
g = gen()
a = next(g)
b = g.send(5)
c = next(g)
```

Завершение

- проверка, не завершен ли генератор
- проверка, не выполняется ли генератор сейчас
- перевод состояния генератора в `FRAME_EXECUTING`
- помещение `None` на стек фрейма
- перевод состояния генератора в `FRAME_CLEARED`
- очистка фрейма (удаление локальных переменных и стека)
- возбуждение `StopIteration`

```
def gen():  
    x = 10  
    y = yield x  
    yield x + y
```

```
g = gen()  
a = next(g)  
b = g.send(5)  
c = next(g)
```

Эволюция

Back to Python 2.2

Генераторы впервые появились в Python 2.2, до этого использовались только **ручные итераторы**.

```
class Counter:
    def __init__(self):
        self.current = 0

    def __iter__(self):
        return self

    def __next__(self):
        result = self.current
        self.current += 1
        return result

counter = Counter()
for i in range(10):
    print(next(counter))
```

```
def counting():
    count = 0
    while True:
        yield count
        count += 1

counter = counting()
print(next(counter))
```

Back to Python 2.2

```
class TreeIterator:
    def __init__(self, root):
        self.stack = [(root, 0)] # (node, state)
    def __iter__(self):
        return self
    def __next__(self):
        while self.stack:
            node, state = self.stack[-1]
            if node is None:
                self.stack.pop()
                continue
            if state == 0:
                self.stack[-1] = (node, 1)
                self.stack.append((node.left, 0))
            elif state == 1:
                self.stack[-1] = (node, 2)
                return node.value
            else:
                self.stack.pop()
                self.stack.append((node.right, 0))
        raise StopIteration
```

```
def tree_traverse(node):
    if node is None:
        return
    for value in tree_traverse(node.left):
        yield value
    yield node.value
    for value in tree_traverse(node.right):
        yield value

for value in tree_traverse(root):
    print(value)
```

Появление yield from

```
def tree_traverse(node):  
    if node is None:  
        return  
    for value in tree_traverse(node.left):  
        yield value  
    yield node.value  
    for value in tree_traverse(node.right):  
        yield value
```

```
for value in tree_traverse(root):  
    print(value)
```

```
def tree_traverse(node):  
    if node is None:  
        return  
    yield from tree_traverse(node.left)  
    yield node.value  
    yield from tree_traverse(node.right)
```

```
for value in tree_traverse(root):  
    print(value)
```

Появление `yield from`

- проксирует `next`: `next(outer) → next(inner)`
- проксирует `send`: `outer.send(x) → inner.send(x)`
- проксирует `throw`: `outer.throw(e) → inner.throw(e)`
- возвращает значение!

```
def subgen():  
    yield 1  
    yield 2  
    return 42  
  
def delegator():  
    value = yield from subgen()  
    print(f"Got: {value}")  
  
g = delegator()  
next(g)    # 1  
next(g)    # 2  
next(g)    # ВЫВОД "Got: 42" + StopIteration
```


Появление `yield from`

Обработка `yield from`:

- получение подгенератора из вершины стека

```
PyObject *subiter = PEEK(1);
```

Появление `yield from`

Обработка `yield from`:

- получение подгенератора из вершины стека
- запрос следующего значения подгенератора

```
PyObject *next = Py_TYPE(subiter)->tp_iternext(subiter);
```

Появление `yield from`

Обработка `yield from`:

- получение подгенератора из вершины стека
- запрос следующего значения подгенератора
- если значение получено:
 - помещение результата на стек
 - перевод состояния генератора в `FRAME_SUSPENDED_YIELD_FROM`
 - отправка результата наружу (как при обычном `yield`)

```
if (next != NULL && !PyErr_Occurred()) {  
    POKE(1, next);  
    frame->generator_state = FRAME_SUSPENDED_YIELD_FROM;  
    JUMPS_TO(fdeopt, YIELD_FROM);  
    return next;  
}
```

Появление `yield from`

Обработка `yield from`:

- получение подгенератора из вершины стека
- запрос следующего значения подгенератора
- если значение получено:
 - помещение результата на стек
 - перевод состояния генератора в `FRAME_SUSPENDED_YIELD_FROM`
 - отправка результата наружу (как при обычном `yield`)
- если `StopIteration`:
 - получение `value` из `StopIteration`
 - замена в стеке: подгенератор → полученное `value`
 - продолжение алгоритма в delegator

Появление `yield from`

```
if (PyErr_ExceptionMatches(PyExc_StopIteration)) {  
    PyObject *exc = _PyErr_GetRaisedException();  
    // получение value из StopIteration  
    PyObject *value = ((PyStopIterationObject *)exc)->value;  
    Py_DECREF(exc);  
    PyErr_Clear();  
    // замена в стеке: подгенератор -> полученное value  
    POKE(1, value);  
    // выход из YIELD_FROM  
    JUMPS_TO(fdeopt, AFTER_YIELD_FROM);  
    DISPATCH();  
}
```

- если `StopIteration`:
 - получение `value` из `StopIteration`
 - замена в стеке: подгенератор → полученное `value`
 - продолжение алгоритма в delegator

Корутины

Корутины в Python зародились из генераторов:

- PyCoro_Type имеет ту же структуру что и PyGen_Type
- Python определяет корутину через флаг CO_COROUTINE
- await-команды “под капотом” работают через yield from

```
import asyncio
async def coro():
    await asyncio.sleep(1)
    return 42
```

Корутины

Обработка `await asyncio.sleep(1)` :

- получение `asyncio.sleep` из вершины стека

```
PyObject *awaitable = PEEK(1);
```

```
import asyncio
async def coro():
    await asyncio.sleep(1)
    return 42
```

Корутины

Обработка `await asyncio.sleep(1)`:

- получение `asyncio.sleep` из вершины стека
- проверка, является ли `asyncio.sleep` корутиной (да)

```
if (PyCoro_CheckExact(awaitable)) {  
    // ...  
}
```

```
import asyncio  
async def coro():  
    await asyncio.sleep(1)  
    return 42
```


Корутины

Обработка `await asyncio.sleep(1)`:

- получение `asyncio.sleep` из вершины стека
- проверка, является ли `asyncio.sleep` корутиной (да)
- получение итератора через метод `__await__()`

```
PyObject *method = PyObject_GetAttr(awaitable, &_Py_ID(__await__));  
PyObject *iter = _PyObject_CallNoArgs(method);
```

```
import asyncio  
async def coro():  
    await asyncio.sleep(1)  
    return 42
```

Корутины

Обработка `await asyncio.sleep(1)`:

- получение `asyncio.sleep` из вершины стека
- проверка, является ли `asyncio.sleep` корутиной (да)
- получение итератора через метод `__await__()`
- замена `asyncio.sleep` на итератор в стеке

```
SET_TOP(iter);
```

```
import asyncio
async def coro():
    await asyncio.sleep(1)
    return 42
```

Корутины

Обработка `await asyncio.sleep(1)`:

- получение `asyncio.sleep` из вершины стека
- проверка, является ли `asyncio.sleep` корутиной (да)
- получение итератора через метод `__await__()`
- замена `asyncio.sleep` на итератор в стеке
- обработка инструкции `yield from`

```
import asyncio
async def coro():
    await asyncio.sleep(1)
    return 42
```

Бенчмарки

Генератор vs Список

Особенности:

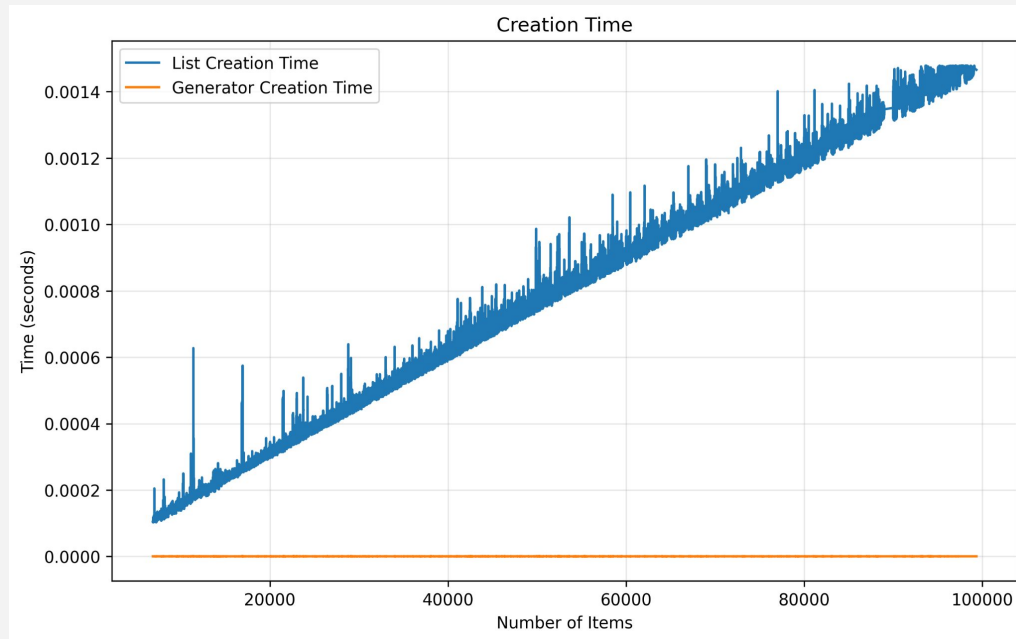
- генератор не вычисляет значения заранее (lazy evaluation)
- список вычисляет всё сразу

Метрики:

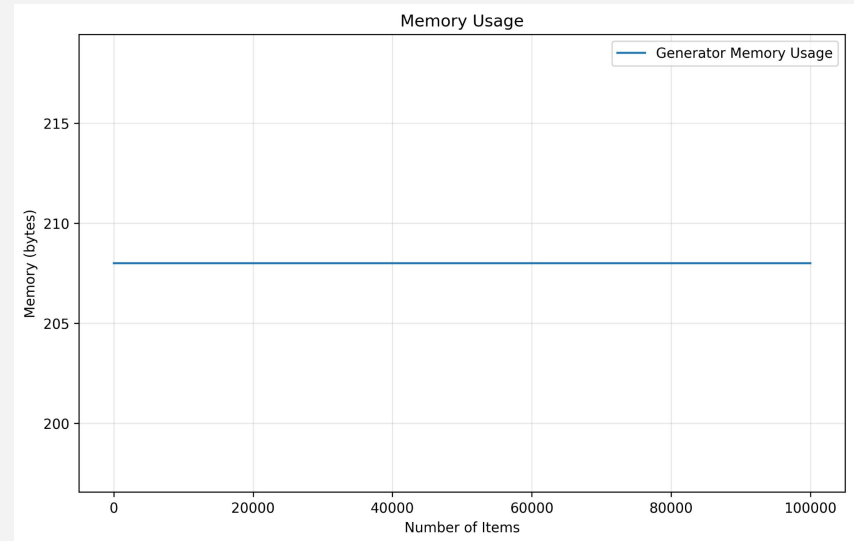
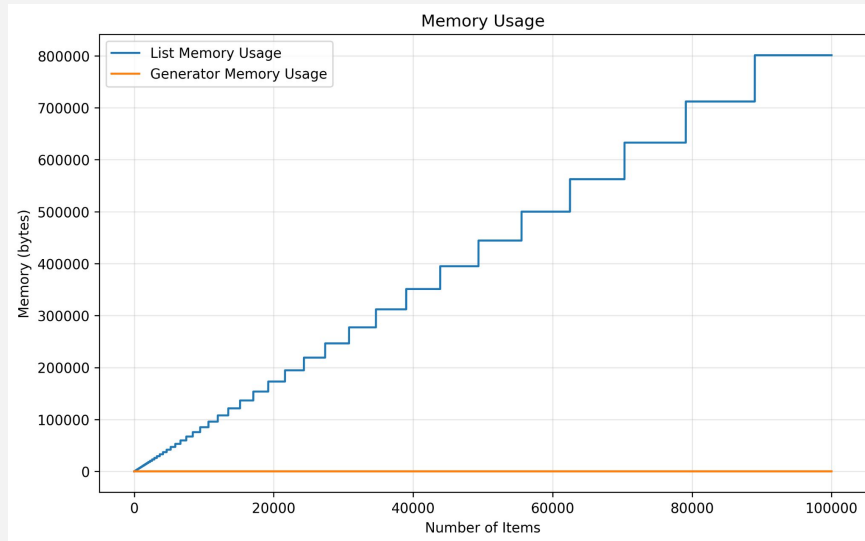
- время создания
- потребление памяти
- время для итерации

```
def list_version(cnt):  
    return [i * i for i in range(cnt)]  
  
def generator_version(cnt):  
    return (i * i for i in range(cnt))
```

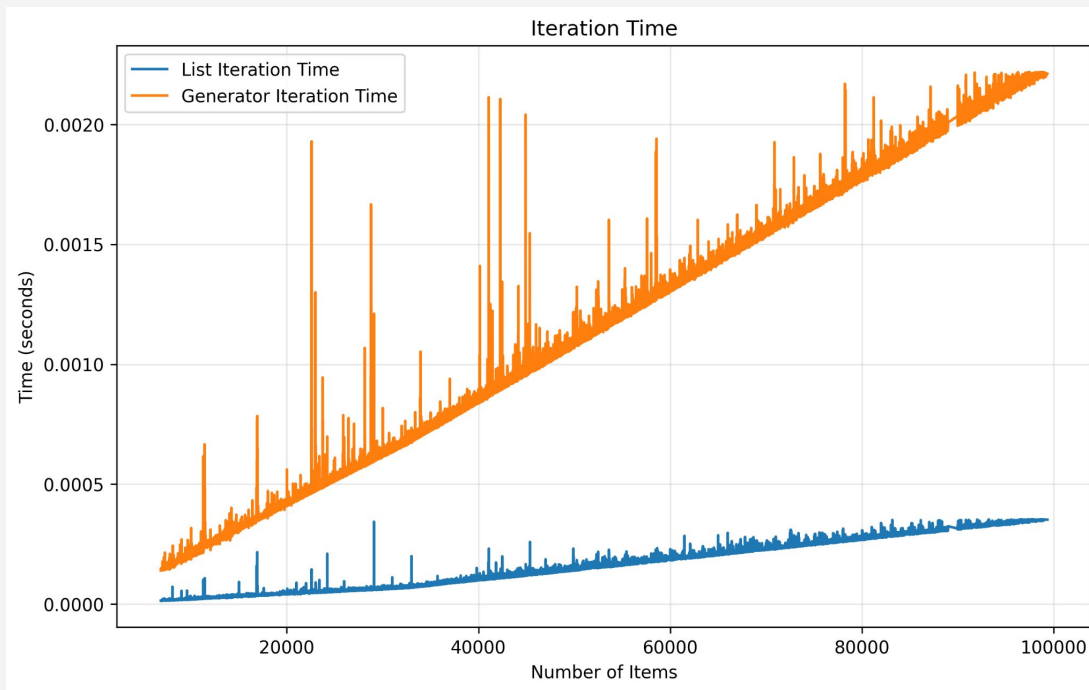
Генератор vs Список: время создания



Генератор vs Список: потребление памяти



Генератор vs Список: время для итерации



Генератор vs Функция

Особенности:

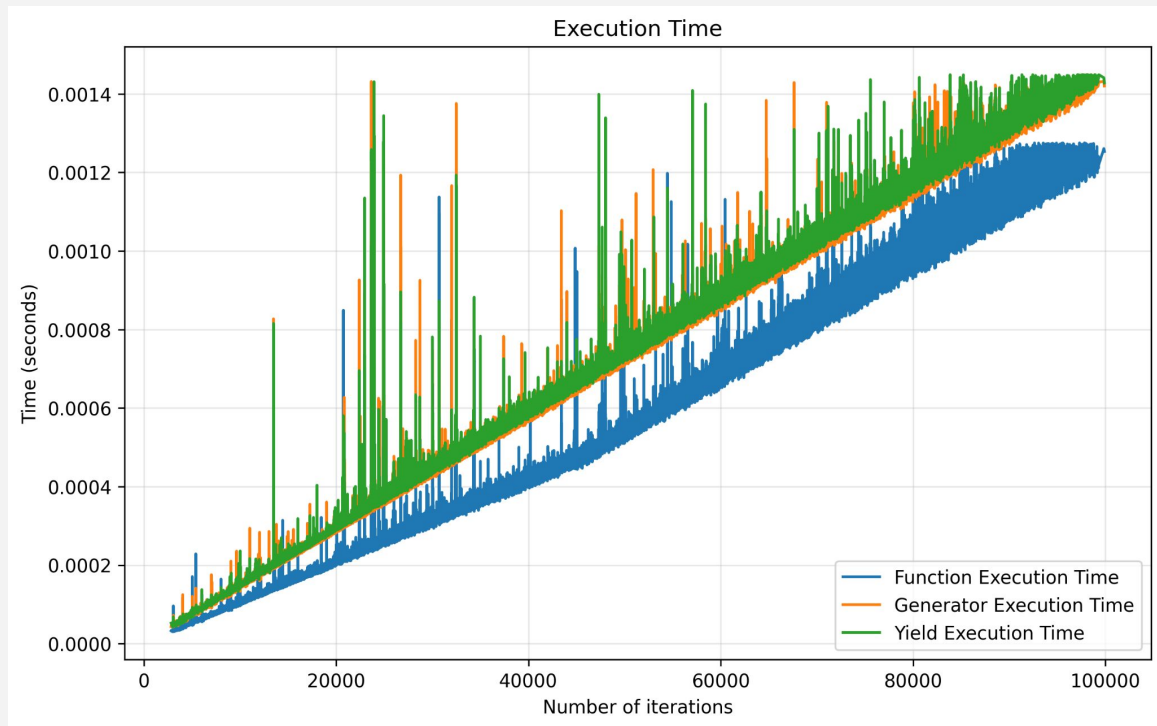
- генератор дороже в использовании, каждый `yield` — переключение контекста
- обычная функция выполняется быстрее

Метрики:

- время выполнения

```
def sum_with_loop(n):  
    total = 0  
    for i in range(n):  
        total += i  
    return total  
  
def sum_with_generator(n):  
    return sum(i for i in range(n))  
  
def gen(n):  
    for i in range(n):  
        yield i  
  
def sum_with_yield(n):  
    return sum(gen(n))
```

Генератор vs Функция: время выполнения



Генератор vs Итератор-класс

Особенности:

- генератор выполняется внутри VM, должен быть быстрее
- итератор требует Python-level вызовов методов, должен быть медленнее

Метрики:

- время выполнения

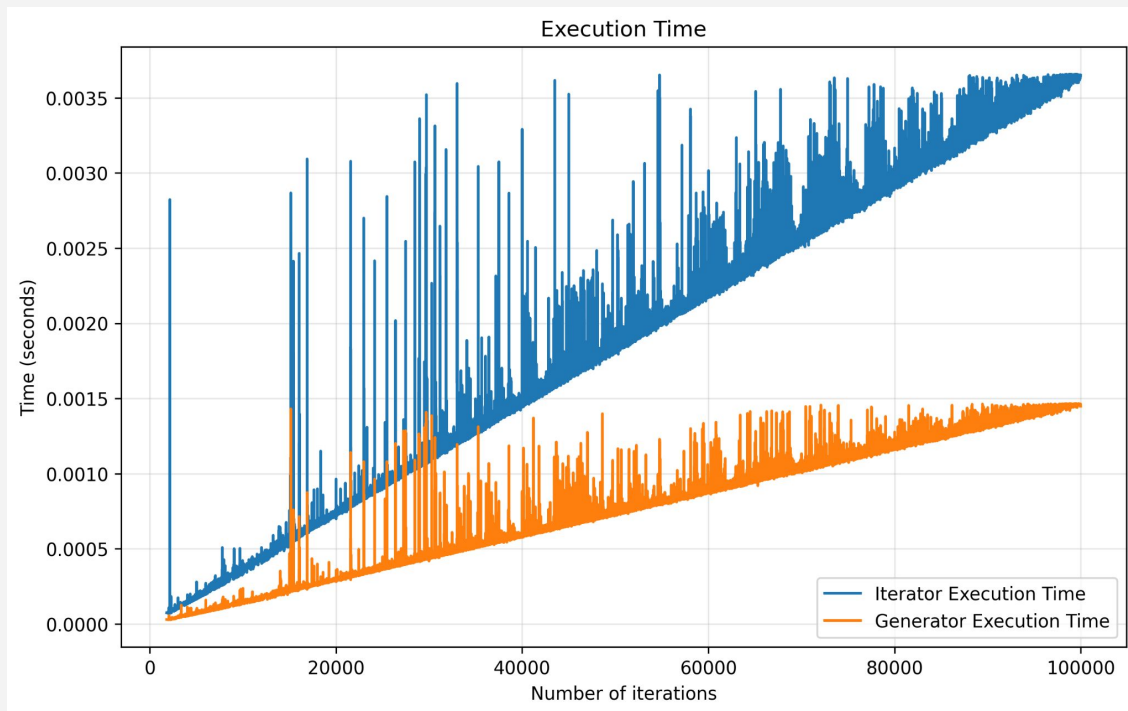
```
class MyIterator:
    def __init__(self, n):
        self.i = 0
        self.n = n

    def __iter__(self):
        return self

    def __next__(self):
        if self.i >= self.n:
            raise StopIteration
        val = self.i
        self.i += 1
        return val

def gen(n):
    for i in range(n):
        yield i
```

Генератор vs Итератор-класс



Накладные расходы на вложенность

Особенности:

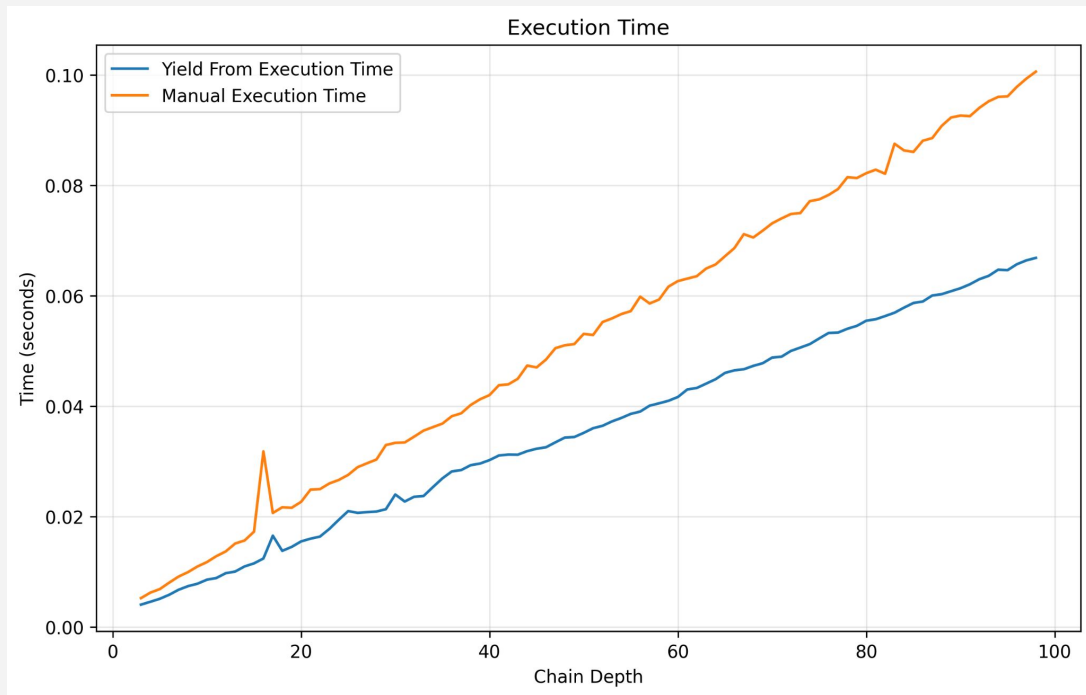
- корреляция глубины вложенности генераторов и производительности
- сравнение ручного делегирования с автоматическим (yield from)

Метрики:

- время выполнения

```
def base_gen(n):  
    for i in range(n):  
        yield i  
  
def make_yield_from_chain(depth):  
    def gen(n, d):  
        if d == 0:  
            yield from base_gen(n)  
        else:  
            yield from gen(n, d - 1)  
    return lambda n: gen(n, depth)  
  
def make_manual_chain(depth):  
    def gen(n, d):  
        if d == 0:  
            for x in base_gen(n):  
                yield x  
        else:  
            for x in gen(n, d - 1):  
                yield x  
    return lambda n: gen(n, depth)
```

Накладные расходы на вложенность



Вопросы